RUDP Video Script (3 speakers, updated for the final 34-slide deck)

Computer Networks, VIT Vellore. Faculty: Anuradha G. Team: Harshavardhan Mallela (24BAI0274), Rohit Arya (24BCE0819), Shloak Sinha (24BDS0378). Due 21 October 2026. Target length about 14 minutes.

Spoken lines are meant to be said aloud: change the wording to suit your voice, but keep the numbers, because each one comes from our own runs.

Speaker	Parts	Slides
Harshavardhan	Opening, what we built, Smart-RUDP, live demo, close	1-6, 15-17, 20, 31-34
Rohit	Reliability, congestion control, literature, neural agent	7-8, 12, 21-25
Shloak	Testing, results, network, Wireshark, final results	9-11, 13-14, 18-19, 26-29

1. Opening and problem (Harshavardhan, 0:00 to 1:15), slides 1-3

Harshavardhan: Hello, I'm Harshavardhan Mallela. **Rohit:** I'm Rohit Arya. **Shloak:** And I'm Shloak Sinha.

Harshavardhan: This is our Computer Networks project, guided by Anuradha G. We built RUDP, a reliable transport protocol, from scratch in C on top of UDP. Then we added a learned congestion controller, designed a four-router OSPF network, and tested all of it.

(slide 2) Packets get lost, duplicated and reordered, and nobody tells you. UDP behaves exactly like that. TCP fixes it, but it lives inside the kernel. In 1986 the internet suffered its first congestion collapse: one path fell from 32 kilobits per second to 40 bits per second. A sender that only retransmits makes congestion worse. So we asked: can we rebuild TCP's guarantees ourselves, and prove they work?

2. What we built (Harshavardhan, 1:15 to 3:00), slides 4-6

Harshavardhan: (slide 4) RUDP has a handshake, sequence numbers, a checksum, retransmission by timeout and by fast retransmit, a sliding window, and congestion control. It runs on plain UDP sockets.

(slide 5) Every packet starts with a 13-byte header: sequence number, acknowledgment number, a flags byte, a checksum and a length. The checksum is a CRC-16. Our first version was a simple byte sum, and a fuzzing test showed that about two percent of corrupted packets slipped through it, so we replaced it. After that, a million random corrupted packets produced zero accepted.

(slide 6) A connection opens with SYN, SYN-ACK, ACK, then data flows with acknowledgments, and FIN closes it. We also fixed a real bug here: if the final handshake ACK is lost, the first data packet now proves the connection is up, as in TCP.

3. Reliability (Rohit, 3:00 to 4:00), slide 7 and 25

Rohit: We started with Go-Back-N: the receiver only accepts the next expected packet. Three duplicate ACKs trigger a fast retransmit. That works, but one loss makes the sender resend a whole window.

(slide 25) So Selective Repeat is now the default. The receiver keeps a 256-packet window and every ACK carries a 32-byte bitmap saying exactly which later packets it holds. The sender then resends only the holes. Go-Back-N is still available with a flag, so we can compare the two.

4. Congestion control (Rohit, 4:00 to 4:45), slide 8

Rohit: For congestion control we implemented AIMD: slow start, then one packet per round trip, halve on fast retransmit, restart on timeout. The sawtooth in this graph is the signature of AIMD coming out of our own code. We also added CUBIC as a second baseline, and an adaptive retransmission timer in the style of RFC 6298.

5. Testing and first results (Shloak, 4:45 to 6:00), slides 9-11

Shloak: (slide 9) We tested with a loss simulator in the client, with Linux netem, and with Wireshark. (slide 10) The benchmark delivered every packet at every loss level from zero to 40 percent; only the time changed. (slide 11) The bugs taught us the most: a server stuck after a killed client, a leftover netem rule that made a healthy run look hung, and Wireshark on Windows not seeing the WSL loopback. Later, real sockets exposed more, which we cover near the end.

6. Network design (Shloak, 6:00 to 7:15), slides 13-14

Shloak: In Packet Tracer we built four routers, R1 to R4, running OSPF in area 0, with two VLANs, DHCP, and an extended ACL that blocks one subnet from another. R2 shows three neighbours in the FULL state. When we shut a link, the route moved through R2 and 70 of 71 pings still got through. Packet Tracer cannot run our C programs, so the network and the protocol are validated separately, and we say so openly.

7. Wireshark (Shloak, 7:15 to 8:00), slides 18-19

Shloak: We wrote a Wireshark dissector in Lua for the RUDP header. Filtering on one sequence number shows every copy of that packet, so retransmissions are visible: the same segment appears again after the timeout.

8. Smart-RUDP (Harshavardhan, 8:00 to 9:30), slides 15-17

Harshavardhan: (slide 15) Smart-RUDP replaces the fixed AIMD rule with a learned policy that, once per round trip, picks one of five actions: shrink, hold, or grow the window. First a Q-learning table, then a small neural network (deep RL) with 1,413 weights, trained with a DQN. (slide 16) The code for the agent is short: five inputs, two hidden layers of 32, five outputs. (slide 17) Against a well-tuned AIMD it was honest at first: roughly equal, not a win. We kept working instead of stopping there.

9. The neural agent in the C transport (Rohit, 9:30 to 11:00), slides 21-24

Rohit: (slide 22) We moved the network into the C sender and gave it a larger window cap, a slow-start phase, pacing, and a safety net. (slide 24) The safety net hands control to AIMD when ten percent of an interval is lost, because the network is least trustworthy under heavy loss. (slide 23) An ablation showed what matters: the larger window cap and pacing do most of the work, and with the old settings the agent is about 14 percent faster than AIMD, not 50. We also fixed chat issues so the web gateway and the terminal chat speak the same protocol.

10. Final results (Shloak, 11:00 to 12:45), slides 26-31

Shloak: (slide 26) The fair comparison is against tuned baselines: AIMD and CUBIC with the same window cap, same transport, same timers, paired seeds. The neural agent is faster than tuned AIMD by 15 percent on a 5G-like path, 49 percent on a lossy path, 52 percent on a satellite-like path and 52 percent on a fast, high-delay path. On the 10 percent loss path it is a tie, not statistically significant. On 100 random held-out paths it needs 0.53 times the time of tuned AIMD and is faster on 89.

(slide 27) Real UDP sockets through an impairment proxy found problems the emulator could not: a lost handshake ACK that hung transfers, and spurious timeouts from a too-tight timer. Both are fixed and covered by tests. (slide 28) We also built a virtual-time version of the same sender, calibrated against the real clock in 15 of 15 cells.

(slide 30, the window figure) On the lossy path AIMD saws between 5 and 15 packets and needs about nine seconds; the agent rides much higher and finishes in four. (slides 29 and 31) Two honest limits. First, fairness: against tuned AIMD on a shared link, the agent takes about 68 percent. We added an optional polite mode that brings it to roughly an even split, at some cost in speed. Second, we sent a 4 MB file over an impaired path and the received hash matched exactly, in 8.9 seconds against 34.6 for AIMD with the old window cap.

11. Live demo (Harshavardhan drives, 12:45 to 13:45), slide 20

Run, in this order: `./bin/server` and `./bin/client 20` (loss recovery), then `./filetransfer_demo.sh 2 lossy` (three policies, identical hashes), then show the chat in two terminals with `./bin/chat 9001 9002 Alice 30` and `./bin/chat 9002 9001 Bob 30`. Check `tc qdisc show dev lo` first so no leftover netem rule is active.

Harshavardhan: On the left the server reassembles the message despite twenty percent loss. Now the file transfer: three policies, same file, same hash, and the neural agent finishes in about half the time.

12. Learnings and close (all three, 13:45 to 14:30), slides 32-34

Harshavardhan: We learned that measuring honestly matters more than winning. Some early margins were inflated by a timer bug that hurt the baseline more, and we corrected them. **Rohit:** A learned controller helps when the path is lossy or long, ties when loss is heavy, and needs a safety net. **Shloak:** Next steps are kernel netem and multi-machine tests, a fairness-aware training objective, and online adaptation.

All: The code is on GitHub at Capsturrrr/rudp. Thank you.

Recording checklist

- Run `make test` once before recording so the green output is ready if you want to show it.
- Use the 34-slide deck `RUDP_Presentation.pptx`; the numbers above match slides 26-31.
- Do not claim the agent is fair or tested on a real network; say what the evidence shows.